INT 3404: IMAGE PROCESSING

Assignment 1: The Digital Camera Pipeline

Student Name: Dang Dinh Khang
Student ID: 23020675

October 14, 2026

1 Introduction

Digital camera sensors capture physical scene radiance by accumulating photons on an array of photodiodes overlaid with a Color Filter Array (CFA). These raw measurements do not form a directly viewable image: they are recorded in a camera-specific, device-dependent coordinate system, contain substantial dark current offsets, exhibit a sparse single-color-per-pixel mosaic, and behave strictly linearly with respect to incident photon flux.

In this assignment, I implement the complete **Digital Camera Pipeline** (Image Signal Processor - ISP) from scratch in Python to transform the raw sensor data of `campus.CR2` (captured with a Canon EOS 5D Mark III DSLR) into an accurate, aesthetically pleasing, and standardized sRGB image. The sequential pipeline includes:

1. **Raw Image Extraction**: Extracting metadata and pure linear sensor measurements using `dcraw`.
2. **Linearization**: Normalizing pixel values by eliminating dark current offsets and mapping to $[0, 1]$.
3. **Bayer Mosaic Masking & Phase Identification**: Establishing color channel masks and determining the correct Bayer phase.
4. **White Balancing**: Correcting illuminant color casts via four standard algorithms.
5. **Demosaicing**: Reconstructing a dense three-channel RGB representation using 2D bilinear convolution kernels.
6. **Color Space Correction**: Transforming device-dependent Camera RGB coordinates to standard sRGB via CIE XYZ 1931 space.
7. **Brightness Scaling & Gamma Encoding**: Adjusting exposure to target perceptual middle gray and applying non-linear sRGB gamma encoding.
8. **Image Compression**: Evaluating lossless PNG versus lossy JPEG compression across quality factors.

2 Raw Image Extraction

2.1 Reconnaissance

Before performing any image operations, I inspected the sensor-specific metadata using the command-line utility `dcraw` [1]:

```
dcraw -4 -d -v -w -T campus.CR2
```

The output produced by `dcraw` is:

```
Loading Canon EOS 5D Mark III image from campus.CR2 ...
Scaling with darkness 2048, saturation 15488, and multipliers 1.858398 1.000000 1.723633
    1.000000
```

From this output, the critical calibration parameters are identified:

- **Camera Model:** Canon EOS 5D Mark III
- **Darkness (Black Level):** 2048
- **Saturation (White Level):** 15488
- **White Balance Multipliers ($r_{\text{scale}}, g_{\text{scale}}, b_{\text{scale}}, g_{\text{scale}}$):** $[1.858398, 1.000000, 1.723633, 1.000000]$

2.2 Generating the Linear TIFF

Using `dcraw -4 -D -T campus.CR2`, I generated a linear 16-bit TIFF file, preserving the completely unscaled, un-interpolated raw sensor data. This TIFF file serves as the input for the subsequent steps in the camera pipeline. My implementation of the `load_raw_tiff` function is as follows:

```
def load_raw_tiff(self):
    raw = io.imread(self.tiff_path)
    self.raw_image = raw.astype(np.float64)

    h, w = self.raw_image.shape[:2]
    min_val = self.raw_image.min()
    max_val = self.raw_image.max()
    approx_bpp = np.log2(max_val + 1)

    print(f"Image Width: {w}, Image Height: {h}")
    print(f"Min raw value: {min_val}, Max raw value: {max_val}")
    print(f"Approximate bits per pixel: {approx_bpp:.2f} (container: {raw.dtype})")
    return self.raw_image
```

The resulting **raw_image** attributes are:

- **Image Dimensions:** 5796×3870 (Width $\times$ Height)
- **Min Raw Value:** 2029.0
- **Max Raw Value:** 16383.0
- **Bit Depth:** 14 bits per pixel, since $\log_2(16383 + 1) = 14.00$. The TIFF file merely stores these 14-bit sensor values inside 16-bit words (`uint16`); the two most significant bits are never used.

3 Linearization

Silicon photodiodes generate thermal electrons even in complete darkness, producing a non-zero signal offset known as *Dark Current*. Conversely, when photodiode electron wells reach full capacity, saturation occurs. To ensure recorded digital numbers accurately correspond to physical scene radiance, the raw readings P_{raw} must be linearized and normalized onto $[0.0, 1.0]$:

$$P_{\text{linear}} = \text{clip}\left(\frac{P_{\text{raw}} - \text{black}}{\text{white} - \text{black}}, 0.0, 1.0\right) \tag{1}$$

where black $= 2048$ and white $= 15488$. Values ≤ 2048 represent absolute black (0.0), values ≥ 15488 represent sensor saturation (1.0), and intermediate intensities scale linearly.

```
def linearize(self):
    if self.raw_image is None:
        raise ValueError("Raw image is not loaded.")
    p_linear = (self.raw_image - self.black) / (self.white - self.black)
    self.linear_image = np.clip(p_linear, 0.0, 1.0)
    return self.linear_image
```

4 The Bayer Pattern

4.1 The Bayer Pattern Design

The Bayer Color Filter Array (CFA) [2] arranges red, green, and blue color filters in a repeating 2×2 tessellation with 50% Green, 25% Red, and 25% Blue pixels. The physical and biological rationale is designed to mimic the Spectral Sensitivity Function of the human eye:

- **Human Perception:** The eye's peak sensitivity lies in the green spectrum ($\sim$ 555 nm), with Medium (M) and Long (L) cone cells making humans far more receptive to green light.
- **Luminance & Detail:** The human visual system (HVS) derives most luminance (brightness) and fine spatial detail from green wavelengths, while Red and Blue primarily contribute to chrominance (color).
- **Sensor Allocation:** In a 2×2 Bayer pattern, allocating 2 cells to Green (50%) and 1 cell each to Red and Blue (25%) maximizes perceived sharpness and dynamic range where human vision is most sensitive.

The function `_build_bayer_masks` constructs the boolean masks of the three channels for any of the four phases:

```
@staticmethod
def _build_bayer_masks(h, w, pattern):
    pattern = pattern.upper()
    mask_r = np.zeros((h, w), dtype=bool)
    mask_g = np.zeros((h, w), dtype=bool)
    mask_b = np.zeros((h, w), dtype=bool)
    channel_masks = {'R': mask_r, 'G': mask_g, 'B': mask_b}
    tile_positions = [
        (slice(0, h, 2), slice(0, w, 2)),
        (slice(0, h, 2), slice(1, w, 2)),
        (slice(1, h, 2), slice(0, w, 2)),
        (slice(1, h, 2), slice(1, w, 2)),
    ]
    for char, (row_slice, col_slice) in zip(pattern, tile_positions):
        if char in channel_masks:
            channel_masks[char][row_slice, col_slice] = True
    return mask_r, mask_g, mask_b
```

4.2 Identifying the Correct Pattern Phase

Because the 2×2 filter repeats across the sensor, the color of the coordinate $(0, 0)$ determines one of four possible pattern phases: `RGGB`, `BGGR`, `GRBG`, or `GBRG`. I used two independent strategies, and both give `RGGB`.

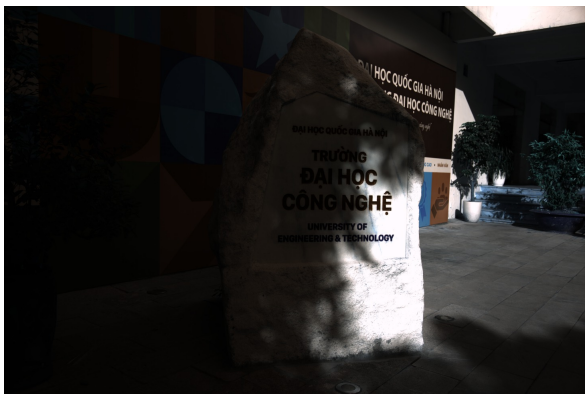
How intermediate images are displayed. Every image shown in Sections 4–7 is rendered without the brightness scaling and gamma encoding of Section 8: the linear values in $[0, 1]$ are written directly as 8-bit pixels ($\times 255$). The images therefore look dark at this point of the pipeline; Section 8 explains why. Each caption states the step after which the image was taken.

4.2.1 Strategy 1: Visual comparison after demosaicing

Each of the four pattern strings is passed to the pipeline, which is run only as far as demosaicing: linearization $\rightarrow$ white balancing (camera presets, as advised in the assignment) $\rightarrow$ bilinear demosaicing. No color space correction or brightness adjustment is applied. Figure 1 shows the four interpolated images.



(a) `RGGB` (correct phase)



(b) `BGGR` (red and blue swapped)



(c) `GRBG` (shifted by one column)



(d) `GBRG` (shifted by one row)

Figure 1: The four Bayer phases, each shown directly after the demosaicing step (white-balanced with the camera presets, no color correction).

- **`RGGB`:** the white stone is neutral, the potted plants are green, and the blue, orange and yellow panels of the mural can be made out. The image is dark because it is still linear, and the colors are muted because it is still in camera RGB, but they are plausible.
- **`BGGR`:** green stays in place, but red and blue are exchanged. The orange panels of the mural become blue and the blue panels become brown/orange, the red inscription turns blue, and the white stone takes a beige tint because the red and blue gains are applied to the wrong channels.
- **`GRBG` and `GBRG`:** the pixels treated as "green" are actually the sensor's red and blue pixels, while the true green samples are sent to the red and blue channels and amplified by the white balance gains. The whole image turns magenta, including the plants, which is impossible for this scene.

4.2.2 Strategy 2: Automatic detection from the bytes of the RAW file

A RAW decoder has to know the Bayer phase before it can demosaic, and it finds it in the RAW file itself. I re-implemented this process as a separate function, `detect_bayer_pattern()`, placed at the end of `assign_code.py`. It reads the bytes of `campus.CR2` directly with plain Python, without any imaging or metadata library. A CR2 file is built on the TIFF format: a short header followed by *Image File Directories* (IFDs), each of which is a list of 12-byte entries (tag number, data type, count, and the value or the offset where the value is stored). The detection takes five steps.

1. **Header.** The first 16 bytes of the file are

   ```
   49 49 2A 00 10 00 00 00 43 52 02 00 80 19 01 00
   ```

   `49 49` ("II") means all numbers are stored little-endian, `2A 00` is the TIFF identifier 42, `10 00 00 00` is the offset of the first IFD (16), `43 52` is the Canon signature "CR" with version 2.0, and `80 19 01 00` is the offset of the IFD that holds the raw sensor data (72064).

2. **Metadata and CFA tag.** IFD0 gives the make ("Canon") and the model ("Canon EOS 5D Mark III"); its EXIF entry (tag `0x8769`) leads to the EXIF IFD, whose MakerNote entry (tag `0x927C`) leads to Canon's private IFD with the model code `0x80000285` (tag `0x0010`). A file can state its CFA layout in tag `0x828E` (TIFF/EP) or `0xA302` (EXIF), but none of the IFDs of `campus.CR2` contains either tag, so the layout is not recorded in the file. Decoders therefore use the layout that Canon's Bayer sensors have at sensor pixel $(0, 0)$. In `dcraw` this is the default code `filters = 0x94949494`, in which every 2-bit field is the color of one tile position ($0 = \text{R}$, $1 = \text{G}$, $2 = \text{B}$). The position of the field for pixel (r, c) is computed with bit operations,

$$\text{color}(r, c) = \big(\texttt{filters} \gg ((((r \ll 1) \,\&\, 14) + (c \,\&\, 1)) \ll 1)\big) \,\&\, 3, \tag{2}$$

so the lowest byte `0x94` $= \texttt{10 01 01 00}_2$, read from the right, gives $(0, 0) = 00 = \text{R}$, $(0, 1) = 01 = \text{G}$, $(1, 0) = 01 = \text{G}$ and $(1, 1) = 10 = \text{B}$: the sensor tile is `RG/GB`.

3. **Sensor size.** The raw IFD points to the raw data, which is stored as a lossless JPEG stream. Its SOF3 marker (`FF C3`) gives a precision of 14 bits, 3950 lines and 2960 samples $\times$ 2 components per line, i.e. a full sensor of 5920×3950 pixels. This also confirms the 14-bit depth found in Section 2.2.

4. **Crop origin.** The image area does not start at sensor pixel $(0, 0)$; the masked border pixels around it are cut away. For a 5920×3950 Canon sensor, `dcraw` cuts 122 columns on the left and 80 rows at the top, which gives exactly the 5796×3870 image written to `campus.tiff`. The camera's own crop is stored in the MakerNote (SensorInfo, tag `0x00E0`): left border 140, top border 96, which gives the 5760×3840 size of the camera's JPEG.

5. **Phase at the image origin.** Shifting the origin by $(\text{top}, \text{left})$ shifts the tile by the same amount modulo 2:

$$\text{color}_{\text{image}}(r, c) = \text{color}_{\text{sensor}}\big((r + \text{top}) \bmod 2,\ (c + \text{left}) \bmod 2\big) \tag{3}$$

An odd column offset would turn `RGGB` into `GRBG`, and an odd row offset into `GBRG`. Here both offsets are even (80 and 122 for `campus.tiff`, 96 and 140 for the camera's crop), so the tile is unchanged and the pattern is `RGGB`.

The core of the implementation (the full, self-contained function is at the end of `assign_code.py`):

```
order = {b"II": "little", b"MM": "big"}[data[0:2]]          # 1. header
def uint(offset, size):
    return int.from_bytes(data[offset:offset + size], order)
ifd0_offset, raw_ifd_offset = uint(4, 4), uint(12, 4)

def read_ifd(offset):                                       # 12-byte entries
    entries = {}
    for i in range(uint(offset, 2)):
        entry = offset + 2 + 12 * i
        tag, typ, count = uint(entry, 2), uint(entry + 2, 2), uint(entry + 4, 4)
        inline = type_size.get(typ, 1) * count <= 4
        entries[tag] = (typ, count, entry + 8 if inline else uint(entry + 8, 4))
    return entries, uint(offset + 2 + 12 * uint(offset, 2), 4)

has_cfa_tag = any(t in d for d in ifds + [exif, makernote]  # 2. CFA tag?
                  for t in (0x828E, 0xA302))

pos = uint(raw_ifd[0x0111][2], 4) + 2                        # 3. SOF3 header
while data[pos:pos + 2] != b"\xff\xc3":
    pos += 2 + int.from_bytes(data[pos + 2:pos + 4], "big")
sensor_w = int.from_bytes(data[pos + 7:pos + 9], "big") * data[pos + 9]
sensor_h = int.from_bytes(data[pos + 5:pos + 7], "big")

def tile_at(row, col):                                      # 2./5. filters code
    return "RGB"[(DEFAULT_FILTERS >> ((((row << 1) & 14) + (col & 1)) << 1)) & 3]
left, top = DCRAW_CANON_MARGINS[(sensor_w, sensor_h)]       # 4. crop origin
pattern = "".join(tile_at(top + r, left + c) for r in (0, 1) for c in (0, 1))
```

Running it on `campus.CR2` prints:

```
Canon / Canon EOS 5D Mark III (ModelID 0x80000285), byte order little
CFAPattern tag present: False
Sensor: 5920 x 3950, 14-bit (SOF3: 3950 lines x 2960 samples x 2 components)
Sensor tile at (0,0): RGGB
Camera crop origin (left, top) = (140, 96) -> RGGB
dcraw crop origin (left, top)  = (122, 80) -> RGGB
```

The function is independent of the pipeline, whose main block keeps `BAYER_PATTERN = "RGGB"`; the function confirms this value. The result agrees with Strategy 1 and with `dcraw` itself, whose identification mode (`./dcraw -i -v campus.CR2`) reports `Filter pattern: RG/GB`.

5 White Balancing

5.1 Implementation

White balancing discounts the color of the ambient illuminant [3] by scaling the Red and Blue channels relative to the Green channel ($s_g = 1.0$). The gains are applied to the Bayer mosaic itself, at the red and blue sites selected by the masks. I implemented four white balancing algorithms:

1. **White World (Max-RGB):** Assumes the brightest value of each channel belongs to a white surface:
$$s_r = \frac{\max(G)}{\max(R)}, \quad s_b = \frac{\max(G)}{\max(B)} \tag{4}$$

2. **Gray World:** Assumes the average spectral reflectance across the scene is achromatic gray:
$$s_r = \frac{\text{mean}(G)}{\text{mean}(R)}, \quad s_b = \frac{\text{mean}(G)}{\text{mean}(B)} \tag{5}$$

3. **Camera Presets:** Uses the multipliers recorded by the camera and reported by dcraw:
$$s_r = \frac{m_1}{m_2} = \frac{1.858398}{1.0} \approx 1.8584, \quad s_b = \frac{m_3}{m_2} = \frac{1.723633}{1.0} \approx 1.7236 \tag{6}$$

4. **Manual White Balance:** Computed from a user-selected neutral patch ($[y_1 : y_2, x_1 : x_2]$):
$$s_r = \frac{\bar{G}_{\text{patch}}}{\bar{R}_{\text{patch}}}, \quad s_b = \frac{\bar{G}_{\text{patch}}}{\bar{B}_{\text{patch}}} \tag{7}$$

```python
def white_balance(self, method='camera', manual_region=None):
    img = self.linear_image.copy()
    h, w = img.shape
    mask_r, mask_g, mask_b = self._build_bayer_masks(h, w, self.pattern)

    if method == 'white_world':
        s_r = img[mask_g].max() / img[mask_r].max()
        s_b = img[mask_g].max() / img[mask_b].max()
    elif method == 'gray_world':
        s_r = img[mask_g].mean() / img[mask_r].mean()
        s_b = img[mask_g].mean() / img[mask_b].mean()
    elif method == 'camera':
        s_r, s_b = self.wb_mults[0] / self.wb_mults[1], self.wb_mults[2] / self.wb_mults[1]
    elif method == 'manual':
        if manual_region is None:
            manual_region = (2400, 2450, 2800, 2850)
        y1, y2, x1, x2 = manual_region
        patch = img[y1:y2, x1:x2]
        s_r = patch[mask_g[y1:y2, x1:x2]].mean() / patch[mask_r[y1:y2, x1:x2]].mean()
        s_b = patch[mask_g[y1:y2, x1:x2]].mean() / patch[mask_b[y1:y2, x1:x2]].mean()

    img[mask_r] *= s_r
    img[mask_b] *= s_b
    self.linear_image = np.clip(img, 0.0, 1.0)
    return self.linear_image
```

5.2 Manual Details: the Selected Patch

For Manual White Balance I selected a 50×50 pixel patch at rows $y \in [2400, 2450)$ and columns $x \in [2800, 2850)$ of the full-resolution image, i.e. `manual_region = (2400, 2450, 2800, 2850)`. It lies on the white marble plaque of the stone monument, on the blank area directly below the English inscription, in the part of the plaque that is in shade. The marble is a neutral surface, and the patch is uniform and far from saturation. Before white balancing its mean linear values are $\bar{R} = 0.0353$, $\bar{G} = 0.0755$, $\bar{B} = 0.0501$, which gives the gains $s_r = \bar{G}/\bar{R} = 2.1419$ and $s_b = \bar{G}/\bar{B} = 1.5059$ (computed from the unrounded means). After Manual White Balance the patch becomes exactly neutral: $\bar{R} = \bar{G} = \bar{B} = 0.0755$.

Figure 2 shows the patch on the Manual White Balance result: (a) marks its position with a red square, and (b) shows the patch itself enlarged, which is now a neutral gray.



(a) Patch location (red square) (b) Patch zoom (8×)

Figure 2: Selected patch for Manual White Balance, `manual_region = (2400, 2450, 2800, 2850)`, shown on the Manual White Balance result. Image taken after the demosaicing step (linearization → manual white balancing → demosaicing, no color correction, brightness or gamma).

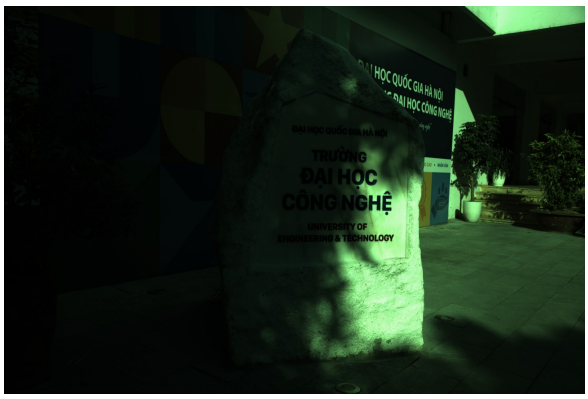
5.3 Comparison

Table 1 lists the gains computed by the four algorithms.

Table 1: Calculated White Balance Scaling Multipliers ($s_g = 1.0$)

Method	$\mathbf{s_r}$	$\mathbf{s_g}$	$\mathbf{s_b}$
White World (Max-RGB)	1.0155	1.0000	1.0155
Gray World	1.7070	1.0000	1.6979
Camera Presets	1.8584	1.0000	1.7236
Manual White Balance	2.1419	1.0000	1.5059

Figure 3 shows the result of each method taken after the demosaicing step: linearization → white balancing → bilinear demosaicing (Section 6). Demosaicing is applied only so that the white-balanced mosaic can be displayed in color; no color correction, brightness scaling or gamma encoding is involved.



(a) White World (Max-RGB)



(b) Gray World



(c) Camera Presets



(d) Manual White Balance

Figure 3: The four white balancing methods. Images taken after the demosaicing step (linearization → white balancing → demosaicing, no color correction, brightness or gamma).

Comparative Analysis:

- **1. White World (Max-RGB):** This method performed the poorest. The overexposed area at the top right of the frame saturates all three channels, so the channel maxima are almost equal ($\max(R) = \max(B) = 0.9847$, $\max(G) = 1.0$) and the gains collapse to $s_r = s_b = 1.0155$. The image is left practically unbalanced and keeps the sensor's native green cast (Figure 3a). The "brightest pixel is white" assumption fails whenever the brightest pixels are clipped.
- **2. Gray World:** The gains ($s_r = 1.7070$, $s_b = 1.6979$) are close to the camera's, and the result is acceptable, but the red gain is about 8% lower than the camera preset, which leaves a slightly cool, green-cyan tint on the white stone. The scene average is not gray: a large part of the frame is covered by the blue and green panels of the mural and by plants.
- **3. Camera Presets:** This produced the most natural result ($s_r = 1.8584$, $s_b = 1.7236$). The sunlit stone and the pavement are neutral and no region shows an obvious cast. The multipliers were measured by the camera for this exposure and do not depend on the content of the scene, so they are not misled by clipped highlights or by a colorful background.
- **4. Manual White Balance:** The selected patch becomes exactly neutral, but the patch is in shade, where the light comes mainly from the blue sky rather than from the sun. Neutralizing it therefore requires a stronger red gain and a weaker blue gain ($s_r = 2.1419$, $s_b = 1.5059$), and the sunlit parts of the stone and the pavement turn noticeably warm (yellowish). The scene has two illuminants, and a single patch can only correct one of them; the result depends strongly on where the patch is taken.

For this scene the Camera Presets are therefore the best choice, and they are used for the rest of the pipeline.

6 Demosaicing

6.1 Implementation

Because each pixel of the Bayer mosaic records only one color channel, the two missing colors must be interpolated [4]. In bilinear demosaicing a missing value is the average of the nearest measured samples of that color. Which neighbors hold such samples depends on the site, so four averaging kernels are needed. Each has a zero center and weights that sum to 1:

$$K_{\text{cross}} = \frac{1}{4}\begin{bmatrix}0 & 1 & 0\\ 1 & 0 & 1\\ 0 & 1 & 0\end{bmatrix}, \quad K_{\text{diag}} = \frac{1}{4}\begin{bmatrix}1 & 0 & 1\\ 0 & 0 & 0\\ 1 & 0 & 1\end{bmatrix}, \quad K_{\text{horiz}} = \frac{1}{2}\begin{bmatrix}0 & 0 & 0\\ 1 & 0 & 1\\ 0 & 0 & 0\end{bmatrix}, \quad K_{\text{vert}} = \frac{1}{2}\begin{bmatrix}0 & 1 & 0\\ 0 & 0 & 0\\ 0 & 1 & 0\end{bmatrix} \tag{8}$$

- **Green at a red or blue site:** the four cross neighbors are all green, so K_{cross} is used.
- **Red at a blue site (and blue at a red site):** the four diagonal neighbors hold the wanted color, so K_{diag} is used.
- **Red or blue at a green site:** only two neighbors hold the wanted color. They are the left and right neighbors if the green site lies in a row that contains that color (K_{horiz}), and the upper and lower neighbors otherwise (K_{vert}).
- **Measured samples** are copied unchanged.

Each kernel is applied with `scipy.ndimage.convolve` to the sparse channel (the mosaic multiplied by the channel mask, so that pixels of other colors contribute zero), and its result is used only at the sites it is meant for. The borders use `mode="mirror"`, which reflects about the center of the edge pixel and therefore keeps the Bayer phase intact outside the image.

```python
def demosaic_bilinear(self):
    img = self.linear_image
    h, w = img.shape
    mask_r, mask_g, mask_b = self._build_bayer_masks(h, w, self.pattern)

    k_cross = np.array([[0, 1, 0], [1, 0, 1], [0, 1, 0]]) / 4.0
    k_diag  = np.array([[1, 0, 1], [0, 0, 0], [1, 0, 1]]) / 4.0
    k_horiz = np.array([[0, 0, 0], [1, 0, 1], [0, 0, 0]]) / 2.0
    k_vert  = np.array([[0, 1, 0], [0, 0, 0], [0, 1, 0]]) / 2.0

    g_sparse = img * mask_g
    g_channel = np.where(mask_g, img, convolve(g_sparse, k_cross, mode="mirror"))

    def interpolate_red_or_blue(mask_c, mask_opposite):
        sparse = img * mask_c
        rows_with_c = mask_c.any(axis=1, keepdims=True)
        g_same_row = mask_g & rows_with_c
        g_other_row = mask_g & ~rows_with_c

        channel = sparse.copy()
        channel[mask_opposite] = convolve(sparse, k_diag, mode="mirror")[mask_opposite]
        channel[g_same_row] = convolve(sparse, k_horiz, mode="mirror")[g_same_row]
        channel[g_other_row] = convolve(sparse, k_vert, mode="mirror")[g_other_row]
        return channel

    r_channel = interpolate_red_or_blue(mask_r, mask_b)
    b_channel = interpolate_red_or_blue(mask_b, mask_r)

    self.rgb_linear = np.clip(np.stack([r_channel, g_channel, b_channel], axis=-1), 0.0, 1.0)
    return self.rgb_linear
```

6.2 Comparison

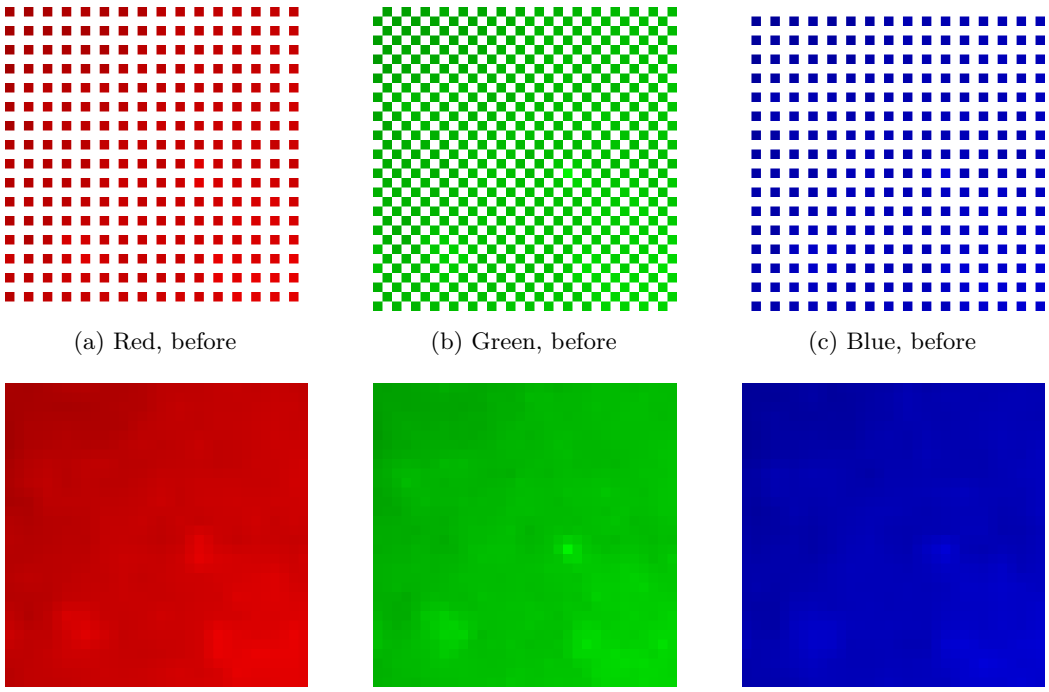
Figure 4 displays the resulting Red, Green, and Blue channels of the image directly after bilinear demosaicing, rendered in their respective primary color planes.



(a) Red channel (b) Green channel (c) Blue channel

Figure 4: The individual R, G, and B channels directly after bilinear demosaicing (rendered in their respective primary color planes).

A full-frame view is reduced too much to show individual pixels, so Figure 5 enlarges a 32×32 pixel window (rows 2464–2495, columns 3368–3399, on the sunlit marble of the plaque just below the inscription, where no value is clipped), each channel drawn in its own color. Before interpolation (top row) a channel holds values only at its own Bayer sites, and the unmeasured sites, shown in white, form a regular grid: 768 of the 1024 pixels are empty for red and blue and 512 for green, which gives the checkerboard. After interpolation (bottom row) every pixel has a value, none of them is zero, and the grid has disappeared.



(a) Red, before (b) Green, before (c) Blue, before

(d) Red, after (e) Green, after (f) Blue, after

Figure 5: A 32×32 pixel window of each channel before (top) and after (bottom) bilinear demosaicing, enlarged 12×. White pixels in the top row are sites where the channel was not measured.

The three channels are dense and consistent with each other, which confirms that bilinear demosaicing reconstructs a complete RGB image from the Bayer mosaic.

7 Color Space Correction

7.1 Implementation

Raw camera sensor dyes do not correspond to the standard sRGB primaries. To map Camera RGB coordinates to standard sRGB, the device-independent CIE XYZ 1931 space is used as an intermediate

bridge:

$$M_{\text{sRGB}\to\text{Cam}} = M_{\text{XYZ}\to\text{Cam}} \cdot M_{\text{sRGB}\to\text{XYZ}} \tag{9}$$

$$M_{\text{Cam}\to\text{sRGB}} = (M_{\text{sRGB}\to\text{Cam}})^{-1} \tag{10}$$

Using the Canon EOS 5D Mark III sensor matrix from the `adobe_coeff` table of `dcraw` and the standard $M_{\text{sRGB}\to\text{XYZ}}$ matrix [5]:

$$M_{\text{XYZ}\to\text{Cam}} = \frac{1}{10000}\begin{bmatrix} 6722 & -635 & -963 \\ -4287 & 12460 & 2028 \\ -908 & 2162 & 5668 \end{bmatrix} \tag{11}$$

$$M_{\text{sRGB}\to\text{XYZ}} = \begin{bmatrix} 0.4124564 & 0.3575761 & 0.1804375 \\ 0.2126729 & 0.7151522 & 0.0721750 \\ 0.0193339 & 0.1191920 & 0.9503041 \end{bmatrix} \tag{12}$$

their product is

$$M_{\text{sRGB}\to\text{Cam}} = \begin{bmatrix} 0.26188661 & 0.18347230 & 0.02519269 \\ 0.09209129 & 0.76195890 & 0.20529817 \\ 0.01948729 & 0.18970602 & 0.53785287 \end{bmatrix}, \qquad \text{row sums: } \begin{bmatrix} 0.47055160 \\ 1.05934836 \\ 0.74704619 \end{bmatrix} \tag{13}$$

The rows of $M_{\text{sRGB}\to\text{Cam}}$ must be normalized before the matrix is inverted: each row is divided by its sum, so that every row sums to 1, and the normalized matrix is then inverted (see Section 7.2). The final matrix applied to every pixel is

$$M_{\text{Cam}\to\text{sRGB}} = \begin{bmatrix} 1.96439218 & -1.11971110 & 0.15531892 \\ -0.24116035 & 1.67375571 & -0.43259536 \\ 0.01388648 & -0.54978120 & 1.53589472 \end{bmatrix} \tag{14}$$

```python
def color_space_correction(self, M_xyz_to_cam):
    M_srgb_to_xyz = np.array([
        [0.4124564, 0.3575761, 0.1804375],
        [0.2126729, 0.7151522, 0.0721750],
        [0.0193339, 0.1191920, 0.9503041]
    ])
    M_srgb_to_cam = M_xyz_to_cam @ M_srgb_to_xyz
    # normalize rows first, then invert
    M_srgb_to_cam = M_srgb_to_cam / M_srgb_to_cam.sum(axis=1, keepdims=True)
    M_cam_to_srgb = np.linalg.inv(M_srgb_to_cam)

    h, w, c = self.rgb_linear.shape
    flat_rgb = self.rgb_linear.reshape(-1, 3)
    corrected_flat = flat_rgb @ M_cam_to_srgb.T
    self.rgb_linear = np.clip(corrected_flat.reshape(h, w, c), 0.0, 1.0)
    return self.rgb_linear
```

7.2 Row Normalization

The rows of $M_{\text{sRGB}\to\text{Cam}}$ must be normalized before the matrix is inverted. The image entering this stage has already been white balanced, so a white surface in it has the camera value $[1, 1, 1]^T$. When every row of $M_{\text{sRGB}\to\text{Cam}}$ sums to 1, sRGB white $[1, 1, 1]^T$ corresponds to exactly this camera white, and the inverse matrix maps camera white back to sRGB white: the rows of $M_{\text{Cam}\to\text{sRGB}}$ in Eq. (14) also sum to 1.

Figure 6 shows the same demosaiced image converted without and with the normalization. Without it, the rows of the inverted matrix sum to $[3.33, 0.49, 1.57]$ instead of $[1, 1, 1]$, so red is amplified and green is suppressed: the white stone and the gray pavement turn pink and the whole image has a strong magenta cast. With the normalization the stone and the pavement stay neutral and the colors of the mural and the plants are natural.

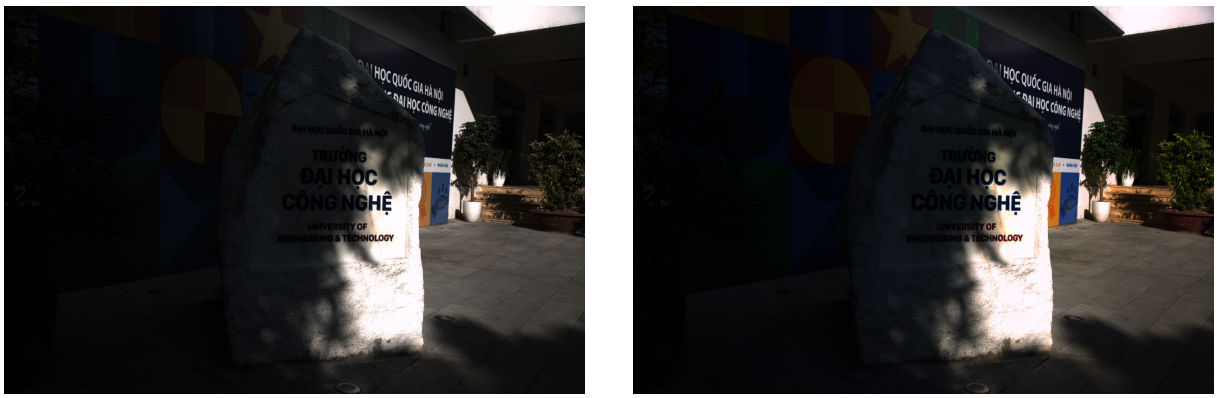


(a) Without normalization (b) With normalization before the inversion

Figure 6: Effect of normalizing the rows of $M_{\text{sRGB}\to\text{Cam}}$ before inverting it.

7.3 Comparison

Figure 7 displays the image immediately before and immediately after color space correction, rendered directly at this pipeline stage in pure linear radiance (without applying the subsequent brightness scaling and non-linear gamma encoding of Section 8).



(a) Before color correction (linear Camera RGB) (b) After color correction (linear sRGB)

Figure 7: Visual comparison before and after Color Space Correction, rendered directly at Step 7 in linear space.

Visual Analysis:

- **Before Color Space Correction (Figure 7a):** The pixels reside in camera-native RGB space. Because the sensor spectral sensitivities differ from standard display phosphor/LCD primaries, chromaticities appear muted, muddy, and desaturated (e.g., foliage has an unnatural olive-yellow cast, and the mural panels appear dull).
- **After Color Space Correction (Figure 7b):** Applying the matrix $M_{\text{Cam}\to\text{sRGB}}$ maps the camera coordinates into the standardized sRGB color space. Foliage shifts into true green chromaticities, the inscription becomes clearly red, and the blue, orange and yellow panels of the mural are faithfully separated. Crucially, neutral white marble and gray pavement remain balanced due to row-sum normalization ($[1, 1, 1]$).
- **Linear Radiance Appearance:** Because both images reflect physical photon counts directly from the sensor without display gamma mapping, they appear visually dark on computer monitors ($\sim$13.4% mean intensity). This directly demonstrates why brightness adjustment and gamma encoding in Section 8 are fundamentally required for human viewing.

8 Brightness and Gamma Encoding

8.1 Implementation

Digital sensors respond linearly to photon counts, whereas human vision is far more sensitive to differences between dark tones than between bright ones. To match human perception:

1. **Brightness Scaling:** The image is scaled linearly so that its mean grayscale intensity becomes 0.25, and clipped to 1:

$$\text{scale} = \frac{0.25}{\text{mean}(\text{rgb2gray}(I))} = \frac{0.25}{0.1345} \approx 1.8588 \tag{15}$$

2. **sRGB Gamma Curve:** Non-linear power curve defined by IEC 61966-2-1 [5]:

$$C_{\text{nonlinear}} = \begin{cases} 12.92 \cdot C_{\text{linear}}, & C_{\text{linear}} \leq 0.0031308 \\ 1.055 \cdot C_{\text{linear}}^{1/2.4} - 0.055, & C_{\text{linear}} > 0.0031308 \end{cases} \tag{16}$$

```python
def brightness_and_gamma(self, target_mean=0.25):
    gray = color.rgb2gray(self.rgb_linear)
    scale_factor = target_mean / gray.mean()
    img_bright = np.clip(self.rgb_linear * scale_factor, 0.0, 1.0)

    mask = img_bright <= 0.0031308
    c_nonlin = np.where(
        mask,
        12.92 * img_bright,
        1.055 * (np.maximum(img_bright, 0.0) ** (1.0 / 2.4)) - 0.055
    )
    self.rgb_srgb = np.clip(c_nonlin, 0.0, 1.0)
    return self.rgb_srgb
```

8.2 Comparison

Figure 8 compares the linear image after brightness scaling (its values written to the file unchanged) with the gamma-encoded image. Although its mean has already been raised to 0.25, the linear image looks far too dark: the shaded side of the stone, the plant on the left and the mural are almost lost, and only the sunlit areas read correctly. A display expects gamma-encoded values, so linear values are shown darker than intended. After gamma encoding the mid-tones and shadows are lifted, the shaded areas become readable, and the highlights are compressed rather than changed.



(a) Linear, after brightness scaling

(b) After sRGB gamma encoding

Figure 8: Comparison between the linear image and the gamma-encoded image.

9 Compression

9.1 Implementation

The floating-point sRGB image in $[0, 1]$ is converted to 8-bit integers (`uint8`) with `img_as_ubyte` from `skimage` and exported as a lossless PNG file and a lossy JPEG file:

```
def save_png_jpeg(self, prefix: str, quality: int = 95):
    img_8bit = img_as_ubyte(np.clip(self.rgb_srgb, 0.0, 1.0))
    png_path, jpg_path = f"{prefix}.png", f"{prefix}.jpg"
    io.imsave(png_path, img_8bit)
    io.imsave(jpg_path, img_8bit, quality=quality)
    png_kb = os.path.getsize(png_path) / 1024.0
    jpg_kb = os.path.getsize(jpg_path) / 1024.0
    ratio = png_kb / jpg_kb
    return png_kb, jpg_kb, ratio
```

9.2 Comparison

The final image was saved as PNG and as JPEG at six quality settings. Table 2 lists the file sizes and the compression ratio (PNG size divided by JPEG size).

Table 2: File size and compression ratio at different JPEG qualities compared to PNG

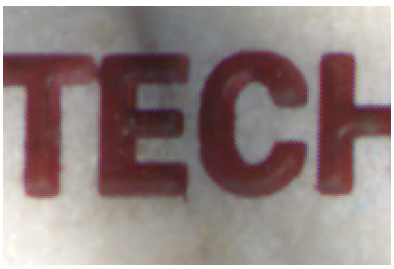
File	File Size (KB)	Compression Ratio
PNG (lossless)	27710.19	1.00×
JPEG, quality 95	4311.16	6.43×
JPEG, quality 85	2034.11	13.62×
JPEG, quality 75	1416.62	19.56×
JPEG, quality 50	897.99	30.86×
JPEG, quality 25	606.93	45.66×
JPEG, quality 10	433.01	63.99×

At the quality required by the assignment (95), the JPEG is 6.43× smaller than the PNG (4.3 MB instead of 27.7 MB).

Compression artifacts cannot be seen in a full-frame reproduction, so Figure 9 shows the same 180×120 pixel region of every file, enlarged 5× without smoothing. The region contains fine marble texture and saturated red letters on a light background, which is where JPEG fails first.



(a) Final image; red frame: region



(b) PNG, 27710 KB



(c) Quality 95, 4311 KB



(d) Quality 85, 2034 KB



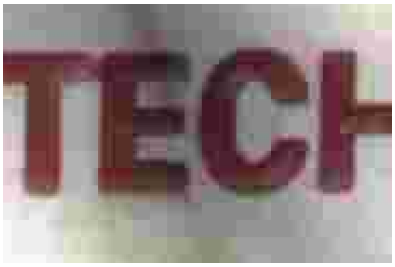
(e) Quality 75, 1417 KB



(f) Quality 50, 898 KB



(g) Quality 25, 607 KB



(h) Quality 10, 433 KB

Figure 9: The same region of the PNG and of each JPEG file, with the size of the whole file.

Lowest Indistinguishable Quality: Comparing each JPEG file with the PNG side by side at normal (100%) size, I could not see any difference down to **quality 50**. Only under the 5× magnification of Figure 9 does quality 50 show a slight smoothing of the fine marble grain, which is invisible at normal size. Below this setting the losses become visible: at quality 25 the 8×8 blocks and the color smearing around the red letters can be seen without magnification, and at quality 10 the image is clearly degraded. I therefore take **quality 50** as the lowest setting that is visually indistinguishable from the PNG. Its file is 897.99 KB against 27710.19 KB for the PNG, a compression ratio of 30.86× (a 96.8% reduction).

10 Statement of AI Usage

In accordance with the assignment guidelines, I used an AI assistant (Google Gemini) as a supporting tool in the following ways:

- **Understanding the theory:** clarifying concepts behind the pipeline, such as why the Bayer pattern has twice as many green pixels, how the sRGB gamma curve is defined, and how the color space correction matrix is derived.
- **Implementation support:** suggestions and debugging help for the more involved parts of the code, in particular the bilinear interpolation kernels in `demosaic_bilinear()`, the row normalization of the color correction matrix, the parsing of the CR2 file in `detect_bayer_pattern()`, and the script that produces the figures.
- **Report formatting:** help with the LaTeX layout of tables, figures, code listings and equations.

All AI suggestions were checked against the lecture material, the assignment specification and the cited references, and every result in this report was verified by running the code.

References

[1] D. Coffin, "Decoding raw digital photos in Linux," https://dechifro.org/dcraw/, 2018, dcraw version 9.28.

[2] B. E. Bayer, "Color imaging array," U.S. Patent 3 971 065, Jul. 20, 1976.

[3] M. Ebner, *Color Constancy*. Chichester, U.K.: John Wiley & Sons, 2007.

[4] H. S. Malvar, L.-w. He, and R. Cutler, "High-quality linear interpolation for demosaicing of Bayer-patterned color images," in *IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, vol. 3, May 2004, pp. 485–488.

[5] International Electrotechnical Commission, "IEC 61966-2-1:1999, Multimedia systems and equipment – Colour measurement and management – Part 2-1: Colour management – Default RGB colour space – sRGB," International Standard, 1999.